

Spartanflow

LinkedIn Content Engine

The Complete Tutorial

Everything you need to build a self-hosted LinkedIn content engine from scratch. Accounts, prerequisites, setup, and step-by-step build.

4-5 hrs

Build Time

\$3-4/mo

Running Cost

6

Workflows

30+

Auto Topics

This document is the companion guide to the Loom video walkthrough. It covers every account, API key, tool, and prerequisite you need before starting the build, plus detailed instructions for each phase.



Hari

CEO & Co-Founder, SpartanFlow

spartanflow.com

Table of Contents

1 What You're Building

Overview of the finished product

2 Accounts & API Keys

Every service you need to sign up for

3 Local Prerequisites

What needs to be installed on your machine

4 Assets to Prepare

Voice profile, headshot, logo, and topic bank

5 Cost Breakdown

Exact monthly costs at 30 posts/month

6 Phase 0: Project Scaffold

Next.js app, Prisma schema, file structure

7 Phase 1: Voice Engine + Humanizer

100-question interview, system prompt, pattern checks

8 Phase 2: Image Generation Pipeline

Gemini API, Sharp compositing, branded images

9 Phase 3: Dashboard UI

Dark theme, PostCards, filters, calendar view

10 Phase 4: Full Pipeline + Polish

End-to-end generation, Slack, ratings, brain dumps

11 Daily Workflow

The 5-minute daily routine

12 Troubleshooting

Common issues and fixes

SECTION 1

What You're Building

A self-hosted web app that handles the entire LinkedIn content pipeline: topic discovery, post generation in your voice, AI-generated branded images, and a dashboard to review everything before posting. No agency. No templates. Just automation.

1. Voice-Matched Writing

- A 100-question voice interview gets turned into a system prompt. A humanizer strips 12 AI patterns. Posts sound like you, not a chatbot.

2. AI Image Generation

- Gemini generates branded images relevant to each post. Your headshot and logo get composited automatically using Sharp.

3. Smart Topic Discovery

- 25+ topics from your real experience, plus trending AI topics from Hacker News API and 8+ RSS feeds. Never run out of ideas.

4. Rating & Learning Loop

- Rate each post 1-5. Log LinkedIn engagement metrics. The AI reads past ratings and learns what resonates.

5. Custom Brain Dumps

- Type raw notes about something you built. The engine turns messy thoughts into a polished LinkedIn post in two minutes.

6. Slack Notifications

- Daily ping: 'Hey, 3 new posts ready for review.' Open dashboard, approve, copy, paste to LinkedIn. Under 5 minutes per day.

SECTION 2

Accounts & API Keys

Every external service you need. Sign up for all of these before starting the build.

Required Accounts

A. Anthropic (Claude API)

- **What it does:** Generates LinkedIn posts in your voice using your system prompt
- **Sign up:** console.anthropic.com
- **You need:** An API key from Settings > API Keys
- **Model used:** Claude Sonnet 4.6 (claude-sonnet-4-6-20250514)
- **Cost:** ~\$0.02 per post (~\$0.60/month at 30 posts)
- **Billing:** Add a credit card and load \$5 minimum. Pay-as-you-go.
- **Env variable:** [ANTHROPIC_API_KEY](#)

B. Google AI Studio (Gemini API)

- **What it does:** Generates branded images relevant to each post
- **Sign up:** aistudio.google.com
- **You need:** An API key from "Get API Key" button
- **Model used:** Gemini 2.5 Flash (image generation)
- **Cost:** ~\$0.04 per image (~\$1.20/month at 30 images)
- **Billing:** Free tier available (limited). For production, enable billing in Google Cloud Console.
- **Env variable:** [GOOGLE_AI_API_KEY](#)

C. Slack (Incoming Webhook)

- **What it does:** Sends you daily notifications when posts are ready for review
- **Sign up:** api.slack.com/apps (you need a Slack workspace first)
- **Setup steps:** Create New App > From Scratch > name it "LinkedIn Engine" > pick your workspace
- **Then:** Go to Incoming Webhooks > Activate > Add New Webhook to Workspace > pick a channel
- **You need:** The webhook URL (starts with [https://hooks.slack.com/services/...](https://hooks.slack.com/services/))
- **Cost:** Free
- **Env variable:** [SLACK_WEBHOOK_URL](#)

Optional Accounts

D. Vercel (Hosting - Optional)

- **What it does:** Hosts your dashboard on the web so you can access it from anywhere
- **Sign up:** vercel.com (sign in with GitHub)
- **You need:** A GitHub account and the repo pushed to GitHub
- **Cost:** Free tier (Hobby plan) handles this easily
- **Alternative:** Run locally with `npm run dev` if you just want it on your machine

E. GitHub Account

- **What it does:** Version control + deployment to Vercel
- **Sign up:** github.com
- **You need:** A repo for the project (Claude Code will create it)
- **Cost:** Free

API Keys Summary

These are the environment variables you'll add to your `.env.local` file:

Variable Name	Source	Required
<code>ANTHROPIC_API_KEY</code>	console.anthropic.com	Yes
<code>GOOGLE_AI_API_KEY</code>	aistudio.google.com	Yes
<code>SLACK_WEBHOOK_URL</code>	api.slack.com/apps	Yes
<code>NEXT_PUBLIC_APP_URL</code>	Your deploy URL or <code>http://localhost:3000</code>	Optional

SECTION 3

Local Prerequisites

What needs to be installed on your machine before you start building.

Required Software

1. Node.js (v18 or later)

- **Why:** Next.js, Prisma, Sharp, and the entire app runs on Node
- **Install:** nodejs.org > Download LTS version
- **Verify:** Run `node --version` in terminal (should show v18+ or v20+)
- **Also installs:** npm (Node Package Manager) comes bundled

2. Claude Code (CLI Tool)

- **Why:** This is the AI coding agent that builds the entire app for you
- **Install:** `npm install -g @anthropic-ai/claude-code`
- **Verify:** Run `claude --version`
- **Requires:** An Anthropic account with Claude Max or API credits
- **Note:** Claude Code is the agent that writes the code. You direct it with prompts.

3. Git

- **Why:** Version control. Claude Code uses git to track changes.
- **Install:** git-scm.com (Mac users: comes pre-installed or via Xcode CLI tools)
- **Verify:** Run `git --version`

4. A Code Editor (Recommended)

- **Why:** To review what Claude Code builds and make manual edits
- **Recommended:** VS Code (code.visualstudio.com) or Cursor
- **Not required:** Claude Code works from the terminal alone, but a code editor helps

5. A Terminal / Command Line

- **Mac:** Terminal.app (built in) or iTerm2
- **Windows:** Windows Terminal, PowerShell, or Git Bash
- **Linux:** Any terminal emulator

Quick Pre-Flight Check

Run these commands in your terminal before starting. All should return version numbers:

```
node --version # Should show v18.x or v20.x+  
npm --version # Should show 9.x or 10.x+  
git --version # Should show 2.x+  
claude --version # Should show latest Claude Code version
```

If any of these fail, install the missing tool before proceeding.

SECTION 4

Assets to Prepare

Before you start building, gather these assets. The build will reference them.

1. Your Voice Profile (Most Important)

- **What:** A 100-question interview about how you write, think, and communicate
- **Why:** This becomes the system prompt that makes AI-generated posts sound like you
- **How:** Use ChatGPT, Claude, or any LLM to interview yourself. Speak your answers out loud using speech-to-text for the most natural voice capture.
- **Guide:** ruben.substack.com/p/i-am-just-a-text-file — full walkthrough on generating your own voice profile
- **Output:** A ~3,500-character system prompt distilled from your full interview
- **Time:** 1-2 hours for the interview, 30 minutes to distill into a system prompt
- **Tip:** Do NOT skip this. This is the single biggest factor in whether the output sounds like you or sounds like a chatbot.

2. Your Headshot (PNG, square crop preferred)

- **What:** A photo of you that gets composited onto every generated image
- **Format:** PNG with transparent background (ideal) or clean background
- **Size:** At least 400x400px. Sharp will handle circle-cropping automatically.
- **Tip:** Use remove.bg (free) to remove the background if needed

3. Your Logo (PNG, transparent background)

- **What:** Your brand logo that gets overlaid on generated images
- **Format:** PNG with transparency
- **Size:** At least 200px wide
- **Placement:** Sharp composites it into a gradient footer bar on each image

4. Your Topic Bank (25-30 topics)

- **What:** A list of real topics from your experience that the engine can write about
- **Format:** Simple list. Title + 1-2 sentence description for each.
- **Categories:** Things you built, lessons learned, tools you use, opinions you hold, your journey
- **Why:** The engine picks from these + trending topics from Hacker News / RSS feeds
- **Tip:** Don't overthink it. "The time I switched from keyword filtering to AI classification" is a perfectly good topic.

SECTION 5

Cost Breakdown

Exact costs at 30 posts per month. This is the full picture.

Service	Monthly Cost	Notes
Claude API (Sonnet 4.6)	~\$0.60	~\$0.02/post x 30 posts
Gemini 2.5 Flash (images)	~\$1.20	~\$0.04/image x 30 images
Hosting (Vercel free tier)	\$0	Free tier handles this easily
Slack webhooks	\$0	Free with any Slack workspace
Hacker News API + RSS	\$0	Public APIs, no key required
SQLite database	\$0	File-based, ships with the app
TOTAL	~\$3-4/mo	Less than a coffee

One-Time Build Costs

Item	Cost	Notes
Claude Code usage (build)	~\$5-15	API costs during the 4-hour build
Claude Max subscription	\$100/mo or \$200/mo	Alternative: use Claude Code with API key directly
Voice interview (your time)	1-2 hours	Free. Use any LLM to conduct the interview.

SECTION 6 — PHASE 0

Project Scaffold

~10 minutes | What Claude Code creates first

This is where you open your terminal, navigate to where you want the project, and give Claude Code its first prompt. Claude Code scaffolds the entire project structure.

What gets created

- Next.js 14 app with App Router (not Pages Router)
- Prisma ORM schema with models for: Posts, Topics, Settings, Ratings, Engagement
- SQLite database file (no external database needed)
- Tailwind CSS configuration with dark theme defaults
- 30+ files: pages, components, API routes, lib utilities
- Package.json with all dependencies
- .env.local template for your API keys

The prompt to give Claude Code

Your initial prompt should describe the full vision so Claude Code can plan the architecture. Include: what the app does (LinkedIn post generator), the tech stack (Next.js + Prisma + SQLite), the features (voice-matched writing, image generation, dashboard, Slack notifications), and that you want it built in phases with verification gates.

Verification gate

Before moving to Phase 1, confirm:

- Dev server runs without errors (`npm run dev`)
- Health check endpoint responds (visit `localhost:3000`)
- Prisma migration completes (`npx prisma db push`)
- Zero build errors in terminal

Voice Engine + Humanizer

~7 minutes | The brain of the system

This is the most important phase. The voice engine is what makes the difference between 'AI-generated LinkedIn slop' and 'posts that actually sound like a human wrote them.' Two components: the voice profile system prompt and the humanizer.

The 100-Question Voice Interview

Before this phase, you should have already completed your voice interview (Section 4). Follow the guide at ruben.substack.com/p/i-am-just-a-text-file to generate yours. The interview covers how you think, write, and communicate across multiple dimensions:

- **Beliefs & worldview** (Q1-Q10): What do you actually do? What's your honest reaction to hype? What role does faith play?
- **Writing pet peeves** (Q14-Q20): What makes you close a tab? What adjectives do you hate? What's your 'kill list' of banned words?
- **Sentence structure** (Q25-Q35): How do you start pieces? How do you end them? Do you use contractions? How does typing vs speaking change your voice?
- **Humor & tone** (Q40-Q50): Is your humor dry, observational, self-deprecating? When do you soften a strong statement?
- **Formatting preferences** (Q55-Q100): How do you emphasize words? How many times do you revise? What does 'done' feel like?

The Humanizer (12 AI Pattern Checks)

After the AI generates a post, the humanizer scans it for patterns that scream 'this was written by a chatbot.' It checks for and removes these patterns:

- Inflated significance ('pivotal moment,' 'testament to,' 'underscoring')
- Promotional language ('groundbreaking,' 'nestled,' 'vibrant,' 'renowned')
- Superficial -ing analyses ('highlighting,' 'showcasing,' 'emphasizing')
- Vague attributions ('experts say,' 'industry observers note')
- Em dash overuse (AI loves em dashes more than humans)
- Rule of three ('innovation, inspiration, and insight')
- Synonym cycling (protagonist/main character/central figure/hero)

- Negative parallelism ('It's not just X; it's Y')
- Sycophantic tone ('Great question!' 'I hope this helps!')
- Excessive hedging ('could potentially possibly be argued')
- Generic positive conclusions ('the future looks bright')
- Inline bold headers with colons in bullet lists

Verification gate

- 29/29 humanizer tests passing
- Voice profile system prompt is 3,000-3,500 characters
- Test generation produces posts that sound like you, not a chatbot

Image Generation Pipeline

~15 minutes | Branded visuals for every post

Each post gets a unique, branded image. Gemini generates the base image based on the post's topic, then Sharp (Node.js image processing library) composites your headshot and logo on top.

How the Pipeline Works

1. Content-aware prompt generation

The system reads the post's topic and category, then generates a specific image prompt for Gemini. A post about 'email deliverability' gets a different visual than 'building in public.'

2. Gemini 2.5 Flash generates the base image

The AI creates a relevant, on-topic image. No stock photos, no generic illustrations.

3. Sharp composites the final image

Your headshot gets circle-cropped and placed in the corner. Your logo goes in a gradient footer bar. The result is a branded, professional image.

Sharp Processing Details

- Headshot circle crop with anti-aliased edges
- Logo overlay with configurable position and size
- Gradient footer bar (semi-transparent dark gradient)
- Output format: PNG at 1200x675 (LinkedIn recommended size)
- All processing happens server-side in the Next.js API route

Verification gate

- Gemini API returns images successfully
- Sharp compositing produces clean output with headshot + logo
- Images render correctly in the dashboard preview

Dashboard UI

~30 minutes | Where you review and manage everything

The dashboard is your command center. Dark theme, responsive, built with Tailwind CSS. This is where you review generated posts, edit them, approve them, and track performance.

Dashboard Components

- **PostCards:** Each post is a card showing the generated text, image preview, topic category, and action buttons: Copy Text, Download Image, Approve, Edit, Rate 1-5.
- **Filter Tabs:** Filter by status: All, Draft, Ready, Approved, Posted. Quickly find what needs your attention.
- **Calendar View:** See your posting schedule at a glance. Which days have posts ready? Which days are empty?
- **Inline Editing:** Click into any post text to edit it directly in the dashboard. No switching to a separate editor.
- **Mobile Responsive:** Works on your phone. Review posts from anywhere.
- **Settings Page:** Configure your voice profile, API keys, Slack webhook, and posting preferences.

Verification gate

- Dashboard loads with 6 sample posts
- All filter tabs work correctly
- Copy, download, approve actions function
- Mobile layout renders properly

Full Pipeline + Polish

~60 minutes | Everything connected end-to-end

This is the longest phase because it connects everything together and adds the advanced features. By the end of this phase, you have a fully working content engine.

What Gets Built

End-to-end generation flow

Click 'Generate' in the dashboard. The system picks a topic (from your bank or trending sources), generates a post in your voice, runs the humanizer, creates an image, and saves everything to the database.

Topic discovery engine

Pulls trending topics from Hacker News API (top stories about AI/tech) and 8+ curated RSS feeds. Combines these with your 25+ personal experience topics for an endless supply of content ideas.

Slack notifications

Sends Block Kit formatted messages to your Slack channel when new posts are ready. Includes post preview, image thumbnail, and a link back to the dashboard.

Rating system

Rate each post 1-5 after reviewing. The system stores your ratings and uses them to improve future generations. Over time, it learns which topics and styles you prefer.

Brain dump feature

Paste raw, messy notes about something you built or learned. The engine transforms your stream-of-consciousness into a polished LinkedIn post. Two minutes from idea to draft.

Engagement tracking

After posting to LinkedIn, come back and log: impressions, likes, comments, reposts. The system tracks which post types perform best.

Verification gate

- Full generation flow works: topic > post > humanize > image > save
- Slack notification arrives with correct formatting
- Rating and engagement logging saves to database
- Brain dump produces a usable post from raw notes

SECTION 11

Daily Workflow

Once built, this is what your day looks like. Under 5 minutes.

1. Slack notification arrives

- Hey, 3 new posts ready for review.

2. Open dashboard

- Glance at generated posts and images.

3. Quick-edit anything

- Fix a word, tweak a line. Inline editing.

4. Approve the good ones

- Hit 'Approve' on posts you like.

5. Copy + download

- Copy text to clipboard. Download the branded image.

6. Post to LinkedIn

- Open LinkedIn. Paste text. Attach image. Post.

7. Log engagement later

- Come back and enter impressions, likes, comments.

SECTION 12

Troubleshooting

Common issues and how to fix them.

Claude Code runs out of context or stalls

Break your prompts into smaller, phase-specific instructions. Don't try to describe the entire app in one prompt. Start a new Claude Code session for each phase if needed.

Gemini API returns errors

Check your API key in `.env.local`. Verify billing is enabled in Google Cloud Console. The free tier has rate limits; if you hit them, wait or upgrade to paid.

Posts sound too AI-generated

Your voice profile needs more specificity. Go back and add more examples of your actual writing. The humanizer catches patterns, but a weak voice profile means the base generation starts too far from your voice.

Sharp compositing fails

Make sure your headshot and logo files are in the correct directory (usually `/public/assets/`). Verify they're PNG format. Check Sharp is installed: `npm ls sharp`.

Slack notifications not arriving

Test the webhook URL directly with curl. If the webhook URL is correct but messages aren't arriving, check the channel permissions in your Slack app settings.

Prisma database errors

Run `npx prisma db push` to sync the schema. If the SQLite file is corrupted, delete `prisma/dev.db` and re-run the push command. You'll lose existing data.

Build errors after Claude Code edits

Run `npm run build` to see the exact error. Claude Code sometimes introduces TypeScript issues. Ask Claude Code to fix the specific error message.

Want something like this for your business?

SpartanFlow builds AI-powered outbound systems that fill your pipeline.

spartanflow.com